Figure 2 visualizes this process. This local search strategy significantly improves the accuracy of entropy estimation by focusing on tokens close to the pivot token. This entire process corresponds to the 'SelectCandidateBatch' function in Algorithm 1.

3.2 Robust Verification of Candidates

Once a batch of promising candidates is selected, each one undergoes a rigorous verification step ('VerifyIsGlitch' in our pipeline) to filter out false positives. A candidate is confirmed as a true glitch only if it fails the repetition task across a set of diverse prompt templates. To ensure this robustness, our verification process uses $m = 3$ templates for cross-validation: in addition to the template used for optimization (Sec. 3.1), we employ two others adapted from prior work (Land and Bartolo 2024a; Li et al. 2024). A token is formally classified as a glitch if and only if its failure rate (as defined in Equation 1) is 1. The additional prompt templates and analysis of false positives are provided in Appendix A.1 and Appendix A.2, respectively.

3.3 Practical Implementation: Search Space Pre-filtering

To maximize search efficiency, following (Land and Bartolo 2024b), we perform a one-time pre-filtering of the vocabulary ('PreFilter' in Algorithm 1). This step removes tokens that pose no real-world risk, allowing GlitchMiner's overall search to focus its resources. We filter three categories:

- **SPECIAL Tokens**: Predefined symbols like `[BOS]` or `</s>`. These are filtered out because they serve a reserved functional role for the model (e.g., indicating the start of a sequence) rather than representing user-generated text.
- **UNDECODEABLE Tokens**: Tokens that correspond to byte sequences that cannot be decoded into a valid string, often because they violate encoding standards like UTF-8. They are excluded as they do not map to any meaningful user-generated text.
- **UNREACHABLE Tokens**: Tokens that exist in the vocabulary but can never be generated by the model's tokenizer from any text input.

Further explanations of these token categories are provided in Appendix A.3.

4 Experiments

4.1 Experimental Setup

Evaluated LLMs. We used a diverse set of LLMs from five different model families to evaluate the performance of our glitch token detection approach. The selected models include Meta's Llama series (Touvron et al. 2023; AI 2024a), Alibaba's Qwen models (Yang et al. 2024; Alibaba 2024), Google's Gemma models (Team et al. 2024), Microsoft's Phi-3 models (Abdin et al. 2024), and Mistral models (Jiang et al. 2023; AI 2024b). The details are presented in Table 1.

Evaluation Metrics. We evaluate our glitch token detection method using the **Detected@N** metric, which counts the number of true glitch tokens identified within the top N predictions. For instance, Detected@1000 measures how many glitch tokens are found among the top 1000 candidates. This metric balances detection accuracy and query efficiency, reflecting a method's practical effectiveness under fixed query budgets. Comparing Detected@N values thus provides a direct measure of each method's ability to maximize glitch token discovery while minimizing computational resources, making it well-suited for real-world applications.

Baselines. We compare our proposed glitch token detection method with two state-of-the-art approaches: GlitchHunter (Li et al. 2024) and Magikarp (Land and Bartolo 2024a). These methods serve as the primary benchmarks for evaluating our approach.

Although GlitchProber (Zhang et al. 2024) is another relevant method, it follows a fundamentally different approach by pre-collecting a subset of glitch tokens to train a classifier, introducing a supervised learning component. In contrast, GlitchMiner, along with GlitchHunter and Magikarp, uses heuristic-based methods to detect glitch tokens without relying on labeled data or additional classifier training. This methodological difference makes a direct comparison less meaningful, so we focus our evaluation on methods that align more closely with our unsupervised approach.

Parameter Settings. In our implementation of GlitchMiner, we use K=32 and B=8 as the default parameters. These values were chosen based on empirical testing to balance computational efficiency and detection effectiveness. Specifically, K=32 defines the size of the local neighborhood considered in each iteration, while B=8 determines the batch size for entropy computation. These settings have shown to provide a good trade-off between exploration of the token space and exploitation of local information across various model architectures.

Initialization Strategy in Experiments. To ensure stable and consistent comparisons across runs, we initialize the search with the token exhibiting the smallest ℓ_2 norm in the embedding space, based on prior observations that such tokens often exhibit glitch-like behaviors. However, as shown in Figure 5, we found that GlitchMiner remains robust to different initialization choices, achieving similar performance even with random starting points.

4.2 Main Results

The performance of GlitchMiner against the baselines is detailed in Table 2. The results provide strong evidence for the effectiveness of our entropy-guided search, showing that GlitchMiner consistently discovers more glitch tokens than both GlitchHunter and Magikarp under fixed query budgets.

On average, GlitchMiner outperforms the strongest baseline, Magikarp, by 10.7% on the Detected@2000 metric. This performance advantage is not uniform but reveals an important trend: while Magikarp's simple norm-based heuristic can be effective for an initial, low-budget scan (e.g., on Llama-3.1-8B), GlitchMiner's more sophisticated local search strategy consistently proves more fruitful as the search budget expands. For instance, on Llama-2-7B-chat-hf, GlitchMiner finds nearly three times as many glitches as Magikarp (532 vs. 186), demonstrating its superior ability to uncover less obvious candidates that simple heuristics miss.

Model Family	Model Names
Llama Models	Llama-3.1-8B-Instruct , Llama-2-7B-chat-hf
Qwen Models	Qwen2.5-7B-Instruct , Qwen2-7B-Instruct
Gemma Models	Gemma-2-2b-it, Gemma-2-9b-it
Phi-3 Models	Phi-3-mini-128k-instruct, Phi-3.5-mini-instruct
Mistral Models	Mistral-7B-Instruct-v0.3, Mistral-Nemo-Instruct-2407

Table 1: Test LLMs used in the experiments.

Model	Metric	GlitchHunter	Magikarp	GlitchMiner (ours)
Llama-3.1-8B-Instruct	Detected@1000	25	**664**	568
	Detected@2000	56	935	**1164**
Llama-2-7B-chat-hf	Detected@1000	61	100	**319**
	Detected@2000	126	186	**532**
Qwen2.5-7B-Instruct	Detected@1000	75	1000	**1000**
	Detected@2000	180	**1893**	1839
Qwen2-7B-Instruct	Detected@1000	96	999	**1000**
	Detected@2000	191	1842	**1847**
Gemma-2-2b-it	Detected@1000	23	678	**744**
	Detected@2000	35	984	**1019**
Gemma-2-9b-it	Detected@1000	29	623	**775**
	Detected@2000	45	983	**1089**
Phi-3.5-mini-instruct	Detected@1000	20	393	**396**
	Detected@2000	44	496	**516**
Phi-3-mini-128k-instruct	Detected@1000	26	398	**404**
	Detected@2000	55	489	**517**
Mistral-7B-Instruct-v0.3	Detected@1000	6	110	**219**
	Detected@2000	19	130	**302**
Mistral-Nemo-Instruct-2407	Detected@1000	48	574	**695**
	Detected@2000	79	918	**976**
Average	Detected@1000	40.9	553.9	**612.0**
	Detected@2000	83.0	885.6	**980.1**

Table 2: Detected@1000 and Detected@2000 comparison of methods across different models.

These findings validate that framing glitch detection as a behavior-driven optimization problem is a more robust and generalizable strategy than relying on static, model-specific patterns.

4.3 Ablation Study

To evaluate the contributions of key components in GlitchMiner, we conducted ablation studies focusing on the local search strategy, neighborhood size K, batch size B, and initialization token.

Effect of Local Search. The local search strategy plays a crucial role in enhancing GlitchMiner's ability to detect glitch tokens by improving the precision of the Taylor approximation. Without local search, detection accuracy drops significantly (Figure 6), as global search lacks the necessary granularity to maintain precise approximations within the token space.

Effect of Neighborhood Size. We analyzed the impact of neighborhood size K on detection performance. As shown in Figure 3, increasing K generally leads to a decline in Detected@1000 values across models. This trend indicates that as K grows, the Taylor approximation becomes less effective, resulting in reduced prediction accuracy.

Effect of Batch Size. As shown in Figure 4, the performance of GlitchMiner remains relatively stable as batch size B increases. Notably, even with $B = 1$, GlitchMiner achieves effective detection results, indicating that it can make accurate predictions without relying on a large batch size.

Effect of Initialization Token. As shown in Figure 5, GlitchMiner's performance remains stable across different initialization tokens. The red dots represent the minimum ℓ_2 norm initialization, while the orange dots show three random trials. For most models, random initialization results are close to the minimum ℓ_2 norm, indicating that GlitchMiner achieves consistent detection accuracy regardless of the initialization approach.

4.4 Token Entropy Analysis

To further validate the effectiveness of our entropy-based approach in detecting glitch tokens, we conducted an entropy analysis comparing glitch tokens and normal tokens across different models. For each model, we computed the average entropy of glitch tokens (E_{Glitch}) and normal tokens (E_{Normal}).

Figure 7 presents the comparison of average entropy values

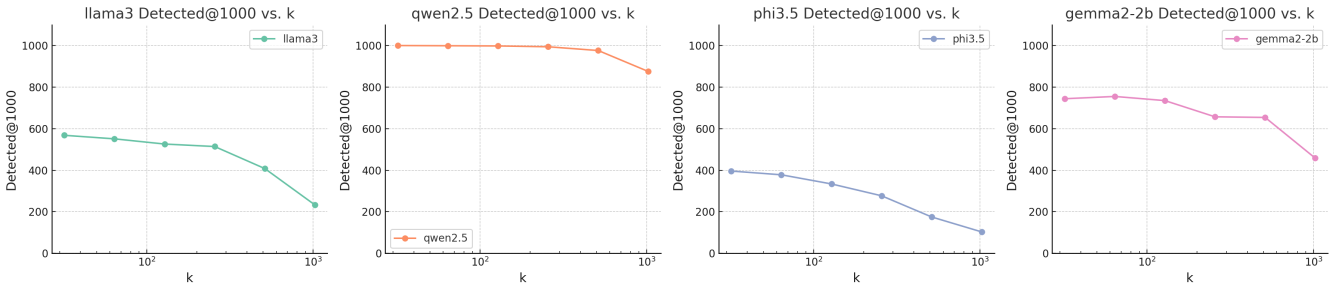

Figure 3: Impact of different Neighborhood Size K on GlitchMiner's performance

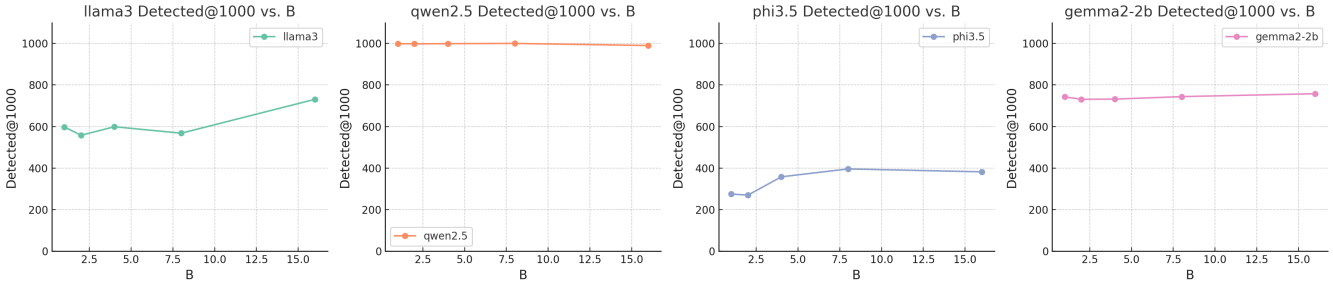

Figure 4: Impact of different Batch Size B on GlitchMiner's performance

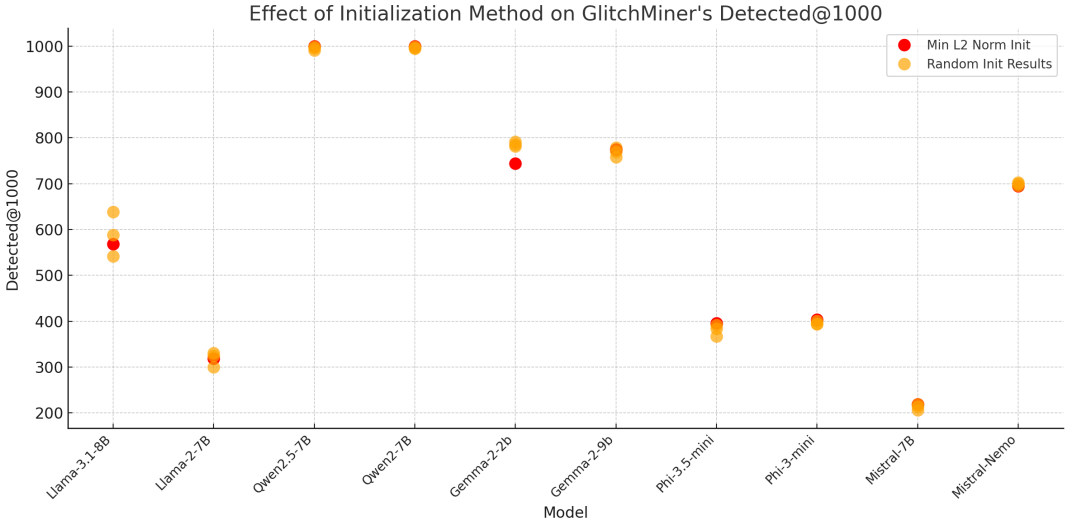

Figure 5: Effect of Initialization Method on GlitchMiner's Detected@1000 score.

between glitch tokens and normal tokens for each evaluated model. As shown in the figure, glitch tokens consistently exhibit significantly higher entropy than normal tokens across all models.

This pronounced difference in entropy values indicates that models are more uncertain when predicting glitch tokens compared to normal tokens. The higher entropy of glitch tokens validates our hypothesis that maximizing entropy effectively guides the search towards tokens that are challenging for the model to predict.

Moreover, the consistent pattern of higher entropy for glitch tokens across diverse model families—including Llama, Qwen, Gemma, Phi-3, and Mistral—demonstrates the generality and robustness of our entropy-based approach. This suggests that our method can be effectively applied to a wide range of LLMs with different architectures and tokenization strategies.

These findings validate the effectiveness of GlitchMiner's entropy-based optimization in efficiently detecting glitch tokens by focusing on areas of high prediction uncertainty